

Instrucciones

- Lea con detenimiento y desarrolle **individualmente** cada una de las actividades a realizar durante la experiencia.
- Cree un archivo con extensión **.zip**, **.rar** o **.tar.gz** que incluya todos los archivos con extensión **.cpp** y **.h** utilizados para el desarrollo. El nombre del archivo debe tener el siguiente formato: **TEL102_C3_Nombre_Apellido.zip** (Ej. TEL102_C3_Patricio_Olivares.zip), sin incluir tildes.
- Las evaluaciones de la asignatura deben ser realizadas de manera **presencial** según lo estipulado en las reglas de clases y evaluaciones de la asignatura.
- Enviar el archivo a través de la página de aula del ramo, sección “Control 3” hasta las 10:45:00 del día Martes 24/10/2023 hora local continental de Chile (UTC-3).
- Trate de utilizar herramientas conocidas o aprendidas en clases. **No copie literalmente de recursos online.**
- Sea riguroso con las instrucciones de desarrollo.
- Si crea nuevos archivos, no olvide agregarlos en su entrega.
- ¡Éxito!

1. Examen de Desarrollo de Aplicación de Gestión de Tareas

La empresa "TaskMaster Inc." ha contratado sus servicios para desarrollar una aplicación de gestión de tareas llamada "TaskPro". Usted debe trabajar en la implementación de diferentes clases y funcionalidades de la aplicación. Los archivos base para el proyecto se encuentran en el aula virtual y son los siguientes:

- main.cpp
- task.h
- tasklist.h
- user.h

El archivo `main.cpp` contiene la lógica principal de la aplicación. A continuación, se muestra parte del código (¡solo puede modificar lo que se indique!):

main.cpp

```
#include "tasklist.h"
#include "user.h"
#include <iostream>

int main() {
    // Creación de un usuario
    User user("Alice");

    // Creación de una lista de tareas
    TaskList taskList("Tareas Personales");

    // Creación de tareas y asignación a la lista
    Task task1("Comprar víveres", "2023-11-01", false);
    Task task2("Hacer ejercicio", "2023-11-05", false);

    taskList.addTask(task1);
    taskList.addTask(task2);

    // Asignación de la lista al usuario
    user.assignTaskList(taskList);

    // Mostrar información de tareas del usuario
    user.showTaskInfo();

    return 0;
}
```

1. (35pts) Complete el archivo `task.h` con la definición e implementación de la clase `Task`. Esta clase representa una tarea y debe contener:
 - Un constructor `Task(std::string description, std::string dueDate, bool isCompleted)`.
 - Atributos para almacenar la descripción de la tarea, la fecha de vencimiento y el estado de completado. Atributos **deben ser privados**.
 - Métodos para obtener y establecer la descripción, la fecha de vencimiento y el estado de completado.
 - Un método para mostrar la información de la tarea, incluyendo su descripción, fecha de vencimiento y estado.
2. (35pts) Complete el archivo `tasklist.h` con la definición de la clase `TaskList`. Esta clase representa una lista de tareas y debe contener:
 - Un constructor `TaskList(std::string name)`. Atributos para almacenar el nombre de la lista y una colección (por ejemplo, un vector) de tareas **deben ser privados**.
 - Métodos para agregar una tarea a la lista, obtener el nombre de la lista y mostrar la información de todas las tareas en la lista, incluyendo su descripción, fecha de vencimiento y estado.
3. (15pts) Complete el archivo `user.h` con la definición de la clase `User`. Esta clase representa un usuario de la aplicación y debe contener:
 - Un constructor `User(std::string username)`.
 - Atributos para almacenar el nombre de usuario y una lista de tareas asignada al usuario. **Atributos deben ser privados**.

- Métodos para obtener el nombre de usuario, asignar una lista de tareas al usuario y mostrar la información de las tareas asignadas al usuario, incluyendo su descripción, fecha de vencimiento y estado.
4. (15pts) Modifique el archivo `main.cpp` para permitir que el usuario marque una tarea como completada. Agregue la funcionalidad para marcar una tarea específica como completada en la/las clases necesarias y muestre la lista actualizada de tareas del usuario.

Asegúrese de completar cada archivo de acuerdo a las especificaciones dadas y de que el programa funcione correctamente al compilarlo.